

PLANEJAMENTO ARQUITETURAL

Concilia IA

Modelo C4 & Architecture Decision Records

Conciliação em Saúde Suplementar — CEJUSC / TJMS

Planejamento Arquitetural — Concilia IA

Planejamento arquitetural **construído do zero** a partir do documento `projeto MVP.pdf`, de forma independente. Descreve a arquitetura da solução (modelo **C4**) e as **decisões arquiteturais (ADRs)** do MVP de uma **API inteligente de apoio à conciliação em saúde suplementar** no contexto do **CEJUSC / TJMS**.

Conteúdo

Documento	Descrição
<code>arquitetura-c4.md</code>	Índice e narrativa do modelo C4 (drivers, RNFs, ligação com ADRs)
<code>c4/</code>	Modelo C4 — um diagrama por arquivo: Contexto, Container, Componente, Dinâmico e Deployment
<code>adr/</code>	Architecture Decision Records — decisões estruturais do MVP

Diagramas do modelo C4 (em `c4/`):

Nível	Arquivo
1 — Contexto	<code>c4/c4-context.md</code>
2 — Container	<code>c4/c4-containers.md</code>
3 — Componente	<code>c4/c4-components-api.md</code>
Dinâmico	<code>c4/c4-dynamic-conciliacao.md</code>
Deployment	<code>c4/c4-deployment.md</code>

Visão geral

A solução — **Concilia IA** — é uma **API REST** que recebe pedidos de conciliação, **consulta as normas da ANS** e os contratos das operadoras, **aplica Inteligência Artificial** para sugerir a melhor estratégia de conciliação e apresenta o resultado ao **conciliador**, que mantém a **decisão final** (*human-in-the-loop*). O objetivo é **reduzir o tempo de análise**, **padronizar as decisões** e **evitar a judicialização desnecessária**.

Pilha tecnológica (do PDF)

Python · FastAPI · PostgreSQL · SQLAlchemy · Alembic · pgvector · Docker · JWT · Swagger/OpenAPI · OpenAI/Claude · GitHub.

Como navegar

1. Comece pelo [modelo C4](#) para entender a estrutura.
2. Consulte os [ADRs](#) para o **porquê** de cada decisão.

Arquitetura da Solução — Modelo C4 (Concilia IA)

Visão geral da arquitetura da **Concilia IA**, a API inteligente de apoio à **conciliação em saúde suplementar** (CEJUSC / TJMS). Os diagramas seguem o **modelo C4** (Simon Brown), em **Mermaid**, com **um diagrama por arquivo** na pasta `c4/`. Documento construído a partir do projeto `MVP.pdf`.

Este arquivo é o **índice e a narrativa** do modelo. Os diagramas estão em:

Nível	Diagrama
1 — Contexto	<code>c4/c4-context.md</code>
2 — Container	<code>c4/c4-containers.md</code>
3 — Componente	<code>c4/c4-components-api.md</code>
Dinâmico	<code>c4/c4-dynamic-conciliacao.md</code>
Deployment	<code>c4/c4-deployment.md</code>

1. Problema e objetivo

Hoje a análise de pedidos de conciliação envolvendo planos de saúde é **manual, demorada**, gera **decisões inconsistentes**, leva a **excesso de judicialização** e há **dificuldade em consultar rapidamente as normas da ANS**.

A **Concilia IA** é uma API que **recebe a solicitação, interpreta o pedido, consulta normas e contratos, aplica IA e sugere a melhor estratégia de conciliação** — sempre com **validação humana** do conciliador.

Metas do MVP: tempo médio de análise inferior a 2 minutos, mais de 80% das sugestões aceitas, redução dos processos encaminhados ao Juizado.

2. Atores e escopo

Ator	Necessidade
Cidadão / Beneficiário	Receber uma solução rápida para o conflito com a operadora
Servidor do CEJUSC	Registrar pedidos com rapidez
Conciliador	Tomar decisões rápidas e fundamentadas
Administrador	Acompanhar estatísticas e desempenho

No escopo: a API REST, suas regras de negócio, persistência, IA e armazenamento de documentos.

Externos: provedores de LLM, base de normas da ANS e os sistemas institucionais do TJMS.

3. Drivers arquiteturais

Driver	Por quê
Confiabilidade da decisão	A IA é probabilística; a decisão final precisa ser humana
Fundamentação / Explicabilidade	Toda sugestão deve citar normas da ANS e cláusulas do contrato
Segurança e privacidade	Dados de saúde são sensíveis
Rastreabilidade / Auditoria	Registrar quem decidiu o quê e com base em quê
Padronização	Reduzir inconsistência entre conciliadores
Reprodutibilidade	Mesmo comportamento em qualquer ambiente

4. Nível 1 — Contexto

A Concilia IA como uma unidade e suas relações com pessoas e sistemas externos (**LLM**, **normas da ANS** e **TJMS**, tratados como caixa-preta).

→ Diagrama: `c4/c4-context.md`

5. Nível 2 — Container

Blocos executáveis e de dados da solução.

→ Diagrama: `c4/c4-containers.md`

Container	Tecnologia	Responsabilidade	ADR
Aplicação Web	SPA Web	Interface das telas do MVP	—
API REST	Python / FastAPI	Endpoints, autenticação, regras, orquestração da IA	0001, 0002, 0003
Banco de Dados	PostgreSQL + pgvector	Persistência, auditoria e busca semântica	0005, 0007
Armazenamento de Documentos	Object Storage (S3)	Contratos e documentos do upload	0010

6. Nível 3 — Componente (API REST)

Estrutura interna da API REST — o container desenvolvido pelo time.

→ Diagrama: `c4/c4-components-api.md`

Componente	Papel	ADR
Autenticação	Login e RBAC via JWT	0004
Cadastros / Solicitações	CRUD e ciclo de vida dos pedidos	0001
Upload de Documentos	Recebe contratos e documentos	0010
Motor de Análise	Orquestra RAG + LLM + sugestão	0008, 0009
Busca Semântica	Recupera normas da ANS e cláusulas	0007
Gateway de LLM	Isola o provedor de IA	0008
Repositórios	Persistência e auditoria	0005, 0012

7. Fluxo principal (pedido → conciliação)

Sequência numerada do registro do pedido até a validação humana. A sugestão nasce **PENDENTE** e só vira decisão após a validação do conciliador; a sugestão original e a decisão são **auditadas**.

→ Diagrama: `c4/c4-dynamic-conciliacao.md`

Decisões: [ADR-0009](#), [ADR-0012](#).

8. Deployment (produção)

Componentes empacotados em **contêineres Docker**; API *stateless* e replicável atrás de *reverse proxy*.

→ Diagrama: `c4/c4-deployment.md`

Decisões: [ADR-0011](#), [ADR-0006](#).

9. Requisitos Não Funcionais

RNF	Como a arquitetura atende
Segurança	TLS, JWT/RBAC, dados sensíveis protegidos (0004)
Confiabilidade da decisão	Validação humana obrigatória (0009)
Fundamentação	RAG com normas da ANS via pgvector (0007)
Auditabilidade	Trilha de auditoria no PostgreSQL (0012)
Portabilidade de IA	Gateway multiprovedor (0008)
Reprodutibilidade	Containerização com Docker (0011)
Evolução do schema	Migrações versionadas com Alembic (0006)

10. Decisões Arquiteturais (ADRs)

O **porquê** de cada decisão está registrado em `adr/` — ver o [índice de ADRs](#).

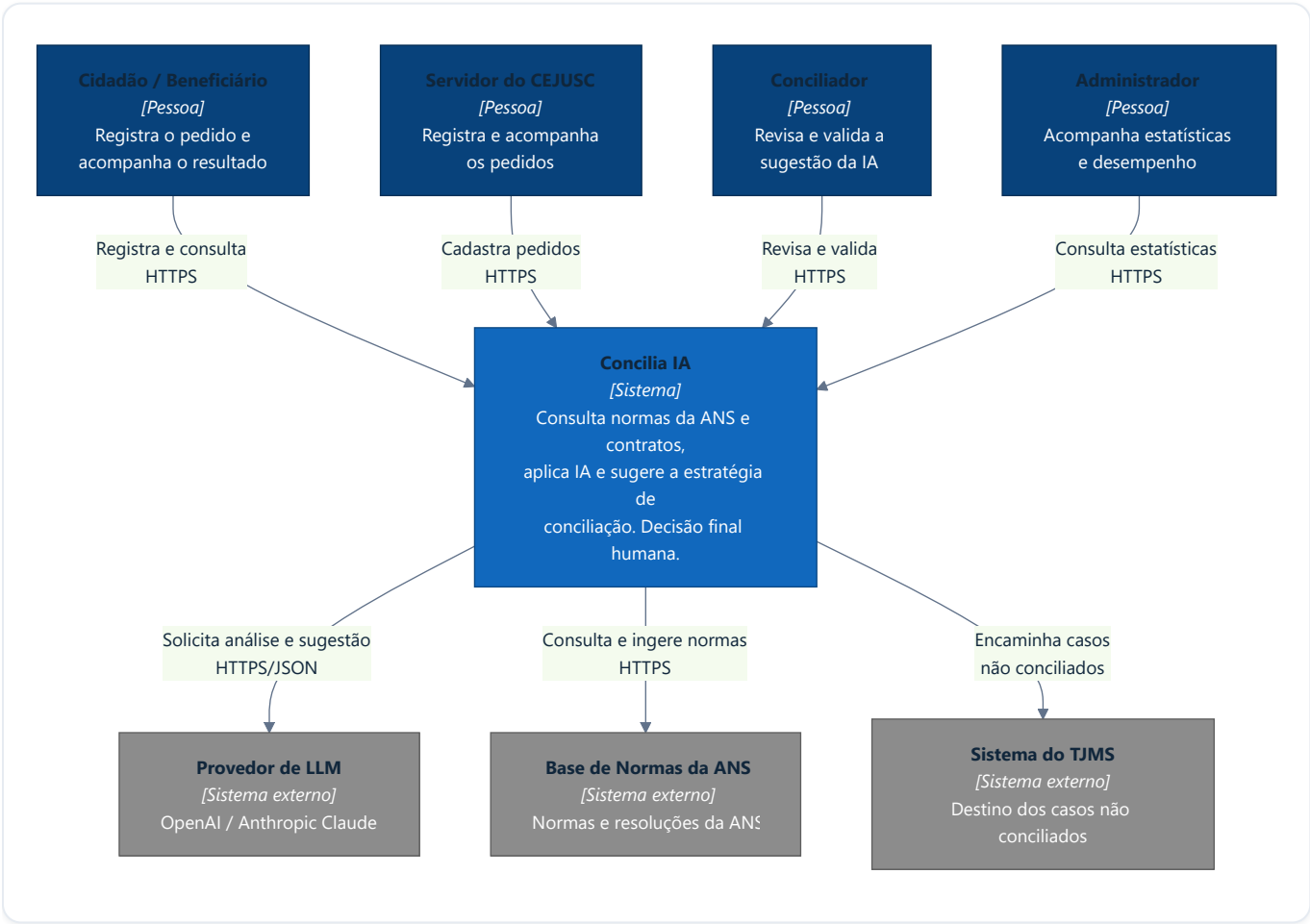
Referências

- Simon Brown — **C4 model** (<https://c4model.com>)
- Documento-base: `projeto MVP.pdf`

C4 — Nível 1: Contexto (Concilia IA)

Mostra a **Concilia IA** como uma unidade e suas relações com pessoas e sistemas externos. Público: todos os interessados.

Notação: **flowchart** seguindo as convenções do C4 (melhor legibilidade que o **C4Context** nativo do Mermaid). Cores: azul-escuro = pessoa, azul = sistema em foco, cinza = sistema externo.



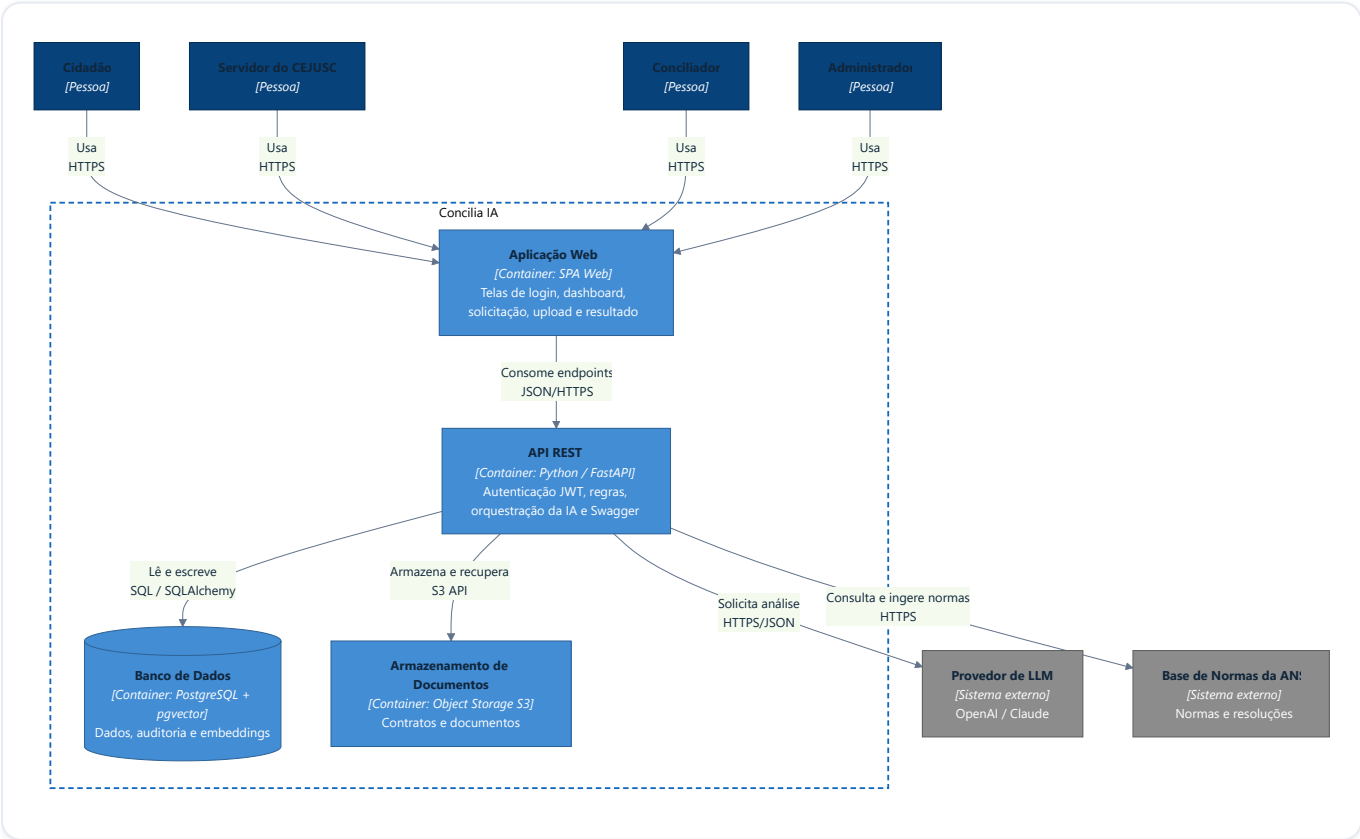
Notas

- **Escopo:** a Concilia IA é o sistema em foco; **LLM**, **normas da ANS** e **TJMS** são externos (caixa-preta).
- A meta institucional é **reduzir o encaminhamento ao Juizado**: casos resolvidos na conciliação não viram processo.
- Decisões relacionadas: [ADR-0008 \(LLM\)](#), [ADR-0009 \(validação humana\)](#).

C4 — Nível 2: Container (Concilia IA)

Detalha os blocos executáveis e de dados da solução. Público: técnico e produto.

Notação: **flowchart** seguindo as convenções do C4. A fronteira tracejada é o sistema **Concilia IA**.



Container	Tecnologia	Responsabilidade	ADR
Aplicação Web	SPA Web	Interface das telas do MVP (consome a API)	—
API REST	Python / FastAPI	Endpoints, autenticação, regras, orquestração da IA	0001, 0002, 0003
Banco de Dados	PostgreSQL + pgvector	Persistência, auditoria e busca semântica	0005, 0007
Armazenamento de Documentos	Object Storage (S3)	Contratos e documentos do upload	0010

Notas

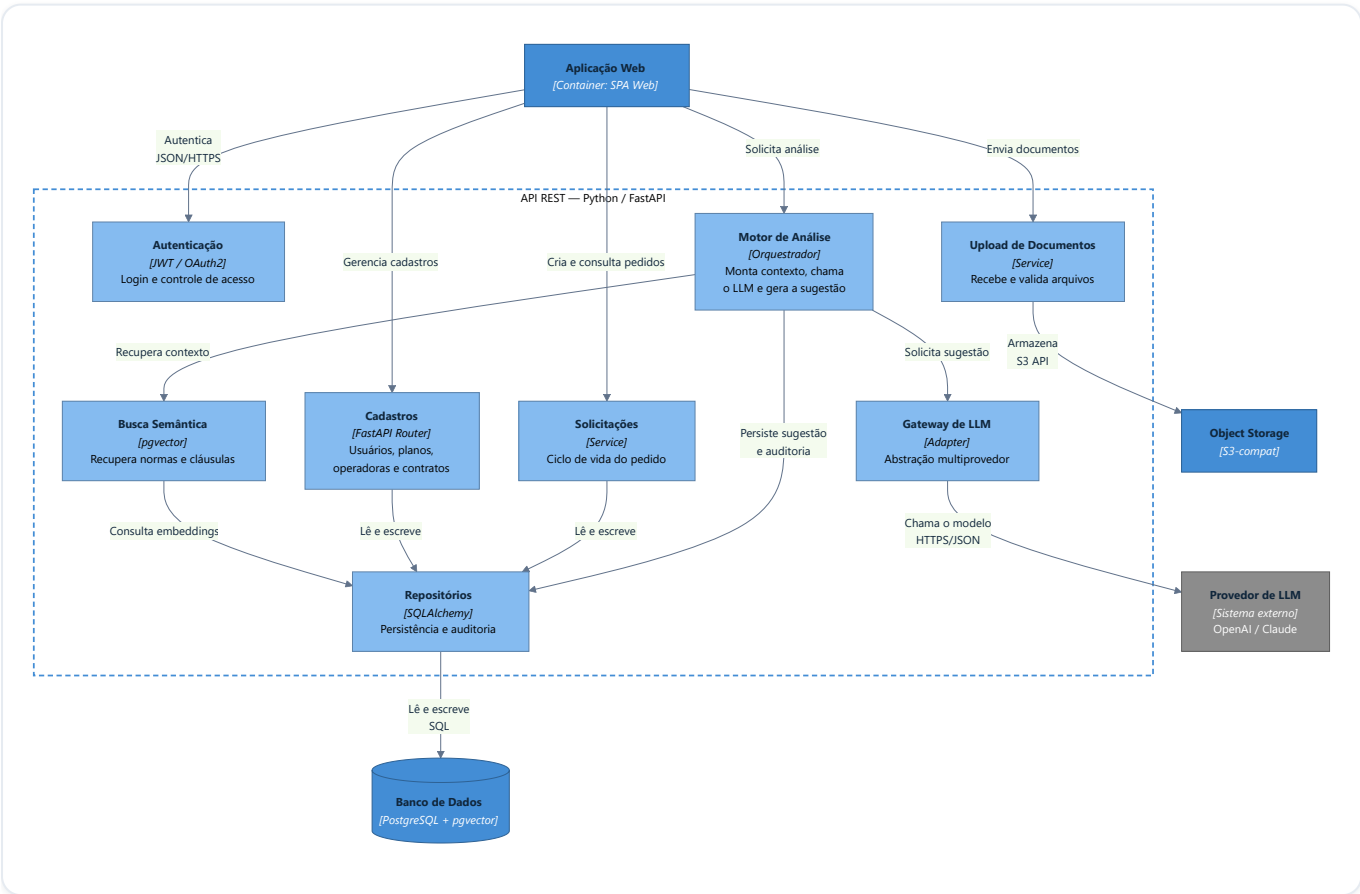
- A **API REST** é única (mono-serviço) — ver [ADR-0001](#).

- O **TJMS** não aparece aqui: é relação de contexto (Nível 1), sem integração técnica em nível de container no MVP.
- Detalhamento interno da API em `c4-components-api.md`.

C4 — Nível 3: Componente (API REST)

Zoom na **API REST**, o container desenvolvido pelo time. Público: desenvolvedores.

Notação: **flowchart** seguindo as convenções do C4. A fronteira tracejada é o container **API REST**; os elementos internos são componentes (não implantáveis isoladamente).



Componente	Papel	ADR
Autenticação	Login e RBAC via JWT	0004
Cadastros / Solicitações	CRUD e ciclo de vida dos pedidos	0001
Upload de Documentos	Recebe contratos e documentos	0010
Motor de Análise	Orquestra RAG + LLM + sugestão	0008, 0009
Busca Semântica	Recupera normas da ANS e cláusulas	0007
Gateway de LLM	Isola o provedor de IA	0008
Repositórios	Persistência e auditoria	0005, 0012

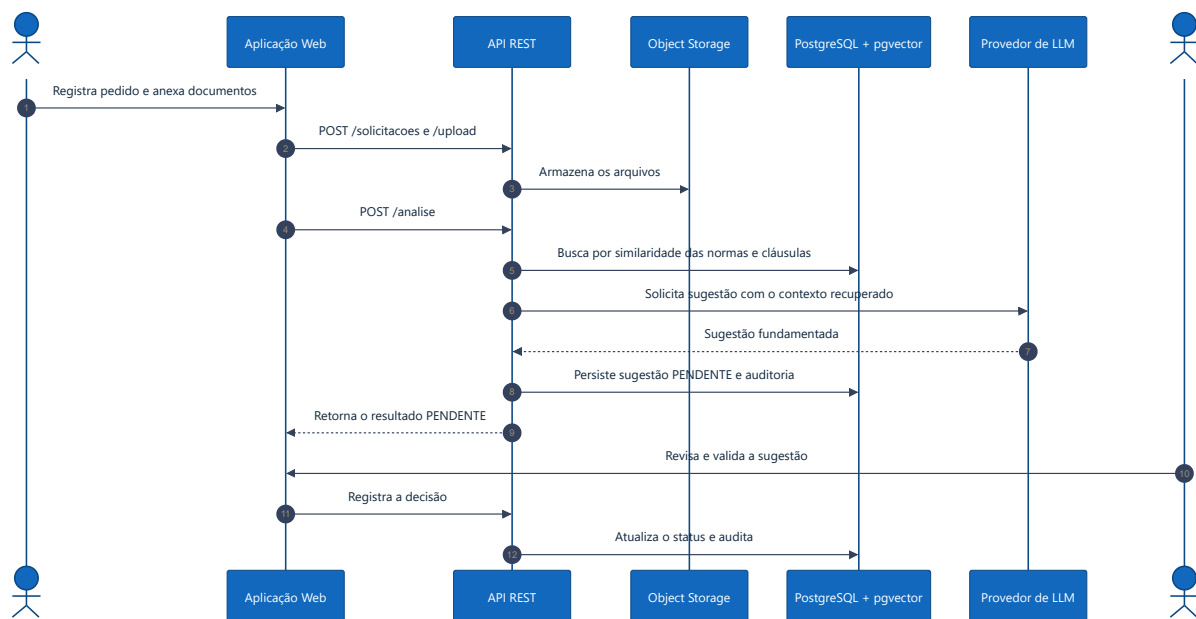
Notas

- Os **componentes** são módulos internos do container **API REST** — não são implantáveis isoladamente.
- O fluxo completo em execução está em `c4-dynamic-conciliacao.md`.

C4 — Dinâmico: Fluxo do pedido à conciliação

Sequência numerada do fluxo principal: do registro do pedido até a validação humana da sugestão.
Público: técnico.

Notação: **diagrama de sequência** (visão dinâmica do C4), com passos numerados automaticamente.



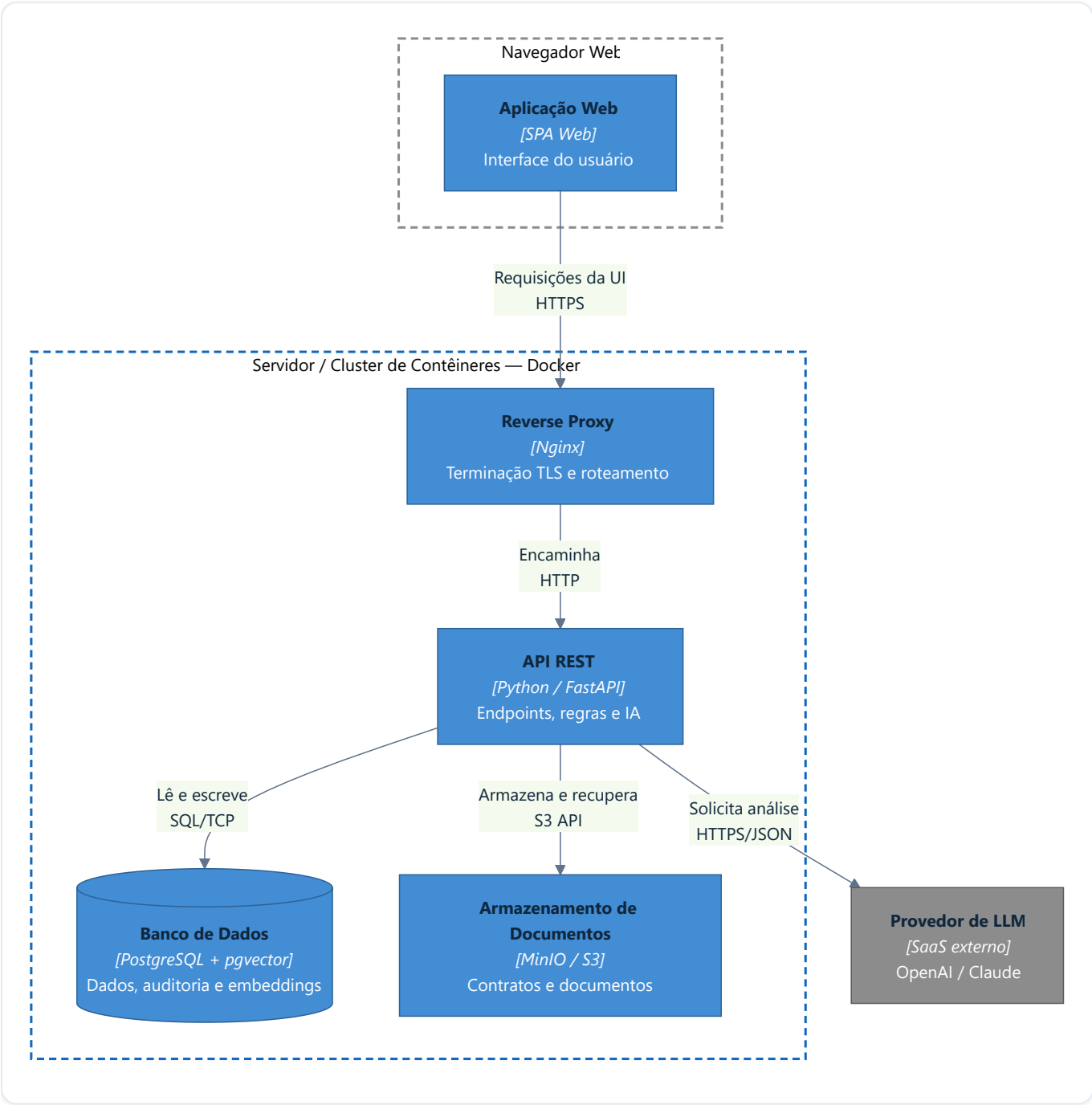
Notas

- A sugestão nasce **PENDENTE** (passo 8) e só vira decisão após a **validação humana** (passos 10–12) — ver [ADR-0009](#).
- O contexto recuperado (passos 5–6) fundamenta a sugestão e é registrado na auditoria — ver [ADR-0007](#) e [ADR-0012](#).

C4 — Deployment: Produção (Docker)

Infraestrutura de implantação em **produção**, com os componentes empacotados em **contêineres Docker**. Público: DevOps.

Notação: **flowchart** seguindo as convenções do C4. Cada caixa tracejada é um nó de implantação (*deployment node*).



Notas

- **Empacotamento com Docker** — ver [ADR-0011](#). Em desenvolvimento, os mesmos serviços sobem via **Docker Compose**.
- A **API é *stateless*** e pode ser replicada horizontalmente atrás do *reverse proxy*; o estado fica no **PostgreSQL** e no **object storage**.
- **Migrações Alembic** rodam como passo do *deploy*, antes de subir a API — ver [ADR-0006](#).

Architecture Decision Records (ADRs) — Concilia IA

Registro das decisões arquiteturais do MVP **Concilia IA**, construídas a partir do documento `projeto MVP.pdf`. Cada ADR documenta uma decisão, suas alternativas e consequências, seguindo o padrão MADR.

Status possíveis: `Proposto` · `Aceito` · `Depreciado` · `Substituído por ADR-XXXX`

ADR	Título	Status
0001	Arquitetura como única API REST (mono-serviço)	Aceito
0002	Python + FastAPI como stack da API	Aceito
0003	REST/JSON com documentação OpenAPI (Swagger)	Aceito
0004	Autenticação e autorização via JWT	Aceito
0005	PostgreSQL + SQLAlchemy para persistência	Aceito
0006	Alembic para migração de schema	Aceito
0007	pgvector para busca semântica das normas da ANS (RAG)	Aceito
0008	Integração multiprovedor de LLM (OpenAI/Claude)	Aceito
0009	Validação humana obrigatória (human-in-the-loop)	Aceito
0010	Armazenamento de documentos e contratos (upload)	Aceito
0011	Docker para empacotamento e ambiente reproduzível	Aceito
0012	Explicabilidade e auditoria das decisões de IA	Aceito

Como adicionar um novo ADR

1. Copie o formato de um ADR existente (Status, Data, Contexto, Decisão, Alternativas, Consequências, Notas de implementação, Referências).
2. Numere sequencialmente (`00NN-titulo-curto.md`).
3. Nunca edite a decisão de um ADR aceito: crie um novo ADR que o **substitui** e marque o antigo como `Substituído por ADR-XXXX` .
4. Atualize esta tabela e a seção 9 de `../arquitetura-c4.md` .

ADR-0001: Arquitetura como única API REST (mono-serviço)

Status

Aceito

Data

2026-07-14

Contexto

O MVP precisa **receber a solicitação, interpretar o pedido, consultar normas e contratos, aplicar IA e sugerir a estratégia de conciliação**. É um time pequeno, com prazo de poucas *sprints* e escopo bem delimitado (cadastros, upload, consulta ANS, análise de IA, sugestão e relatório).

É preciso definir a **granularidade** da arquitetura: um único serviço ou vários serviços independentes (microserviços).

Decisão

Construir a solução como uma **única API REST** (mono-serviço), organizada internamente em módulos/componentes coesos (autenticação, cadastros, solicitações, upload, motor de análise). Todo o processamento — inclusive a orquestração da IA — fica concentrado nesse serviço.

Alternativas Consideradas

Alternativa 1: Microserviços

- **Descrição:** Separar autenticação, análise de IA, cadastros etc. em serviços independentes.
- **Prós:** Escala e *deploy* independentes; isolamento de falhas.
- **Contras:** *Overhead* operacional (rede, observabilidade, orquestração) elevado para o tamanho do time e do MVP.
- **Por que rejeitada:** Complexidade injustificada nesta fase.

Alternativa 2: Funções serverless

- **Descrição:** Cada endpoint como função gerenciada.
- **Prós:** Escala automática; custo por uso.
- **Contras:** *Cold start*, limites de execução para chamadas longas de LLM, fragmentação da lógica.
- **Por que rejeitada:** Ruim para orquestração de IA com estado e chamadas longas.

Consequências

Positivas

- Simplicidade operacional: um *deploy*, um *pipeline*, um ponto de observação.
- Menor latência interna (chamadas em processo, sem rede entre módulos).
- Evolução rápida no ritmo do MVP.

Negativas

- Acoplamento interno maior que em microsserviços.
- Todo o serviço escala junto (mesmo que só a análise seja pesada).

Riscos

- Crescimento do escopo tornar o mono-serviço grande → mitigado por **fronteiras de módulo bem definidas** que permitem extrair serviços no futuro.

Notas de Implementação

- Organizar por módulos com contratos internos claros (facilita extração futura).
- Manter a API **stateless** para permitir réplicas horizontais.

Referências

- [arquitetura-c4.md](#) — Seções 5 e 6

ADR-0002: Python + FastAPI como stack da API

Status

Aceito

Data

2026-07-14

Contexto

O núcleo do produto é o **processamento de IA**: integração com LLMs, manipulação de texto (normas, contratos) e classificação. A escolha da linguagem/framework deve favorecer **produtividade, ecossistema de IA maduro** e bom suporte a **I/O assíncrono** (chamadas a LLMs são predominantemente *I/O bound*).

Decisão

Usar **Python** com **FastAPI** para a API REST. Complementos do mesmo ecossistema: **Pydantic** (validação/serialização), **Uvicorn/ASGI** (servidor assíncrono), **SQLAlchemy** (ORM) e um cliente HTTP assíncrono (**httpx**) para as integrações externas.

Alternativas Consideradas

Alternativa 1: Node.js (TypeScript)

- **Descrição:** Stack JavaScript/TypeScript.
- **Prós:** Linguagem única com o front-end; bom desempenho de I/O.
- **Contras:** Ecossistema de IA/LLM menos maduro; SDKs seguem o Python primeiro.
- **Por que rejeitada:** A vantagem de IA pesa mais que a unificação de linguagem.

Alternativa 2: Java (Spring Boot)

- **Descrição:** Stack corporativa robusta.
- **Prós:** Maturidade, tipagem forte, ferramental amplo.
- **Contras:** Mais verboso; ecossistema de IA/LLM menos idiomático.
- **Por que rejeitada:** Menor velocidade de desenvolvimento no MVP.

Consequências

Positivas

- Acesso direto a SDKs oficiais de LLM e ferramentas de NLP.

- FastAPI gera documentação **OpenAPI/Swagger** automaticamente.
- Validação robusta com Pydantic; suporte nativo a `async/await`.

Negativas

- Desempenho de CPU inferior a Go/Rust (irrelevante para carga *I/O bound*).
- Atenção ao GIL em tarefas CPU-bound.

Riscos

- Gestão de dependências/ambiente → mitigada por *lockfile* e imagem de container reprodutível ([ADR-0011](#)).

Notas de Implementação

- Servir com Uvicorn (workers ASGI) atrás de um *reverse proxy*.
- Habilitar a documentação OpenAPI automática do FastAPI.

Referências

- [arquitetura-c4.md](#) — Seções 5 e 6
- [ADR-0003](#) — REST/JSON com OpenAPI

ADR-0003: REST/JSON com documentação OpenAPI (Swagger)

Status

Aceito

Data

2026-07-14

Contexto

A solução é consumida por uma **Aplicação Web** (telas de login, dashboard, nova solicitação, upload, resultado) e, potencialmente, por outros clientes. É preciso definir o **estilo de interface**: protocolo, formato de dados e forma de documentar o contrato. O PDF já indica os endpoints (`POST /login` , `POST /solicitacoes` , `GET /solicitacoes` , `POST /upload` , `POST /analise` , `GET /resultado`) e o uso de **Swagger**.

Decisão

Expor uma **API REST sobre HTTP** com payloads em **JSON**, recursos orientados a substantivos, verbos HTTP semânticos e códigos de status apropriados. O contrato é documentado automaticamente via **OpenAPI/Swagger** (nativo do FastAPI).

Alternativas Consideradas

Alternativa 1: GraphQL

- **Descrição:** Endpoint único com consultas flexíveis.
- **Prós:** Cliente busca exatamente o que precisa.
- **Contras:** Complexidade de servidor, cache e segurança; *overkill* para o MVP.
- **Por que rejeitada:** Recursos bem definidos não justificam a complexidade.

Alternativa 2: gRPC

- **Descrição:** RPC binário sobre HTTP/2.
- **Prós:** Alto desempenho e contrato forte.
- **Contras:** Menor interoperabilidade com navegadores; ferramental extra.
- **Por que rejeitada:** O consumidor principal é web; REST/JSON tem menos fricção.

Consequências

Positivas

- Interoperabilidade ampla; fácil de testar (curl, navegador, Postman).
- Documentação **Swagger** automática e sempre atualizada.
- Ecossistema maduro (auth, cache HTTP, gateways).

Negativas

- *Over/under-fetching* ocasional comparado ao GraphQL.
- Versionamento de contrato exige disciplina.

Riscos

- Mudanças que quebram o contrato → mitigadas por versionamento (`/v1`) e testes.

Notas de Implementação

- Prefixo de versão `/v1` .
- Erros em formato padronizado (ex.: Problem Details, RFC 9457).
- Autenticação via *bearer token* JWT ([ADR-0004](#)).

Referências

- [arquitetura-c4.md](#) — Seção 5
- [ADR-0002](#) — Python + FastAPI

ADR-0004: Autenticação e autorização via JWT

Status

Aceito

Data

2026-07-14

Contexto

A API expõe endpoints (`/login` , `/solicitacoes` , `/upload` , `/analise` , `/resultado`) consumidos por uma Aplicação Web. É preciso **autenticar** os usuários (conciliadores, servidores, administradores) e **autorizar** o acesso a **dados sensíveis de saúde**. A API deve permanecer **stateless** para escalar horizontalmente.

Decisão

Adotar autenticação baseada em **tokens JWT** assinados. O endpoint `POST /login` valida as credenciais e emite um **access token** (curta duração) e um **refresh token**. Os tokens carregam *claims* de identidade e **papel** (RBAC: `conciliador` , `servidor` , `administrador`). Os endpoints protegidos validam a assinatura e os *claims* via dependência do FastAPI. Sem estado de sessão no servidor.

Alternativas Consideradas

Alternativa 1: Sessão server-side com cookie

- **Descrição:** Sessão mantida no servidor, referenciada por cookie.
- **Prós:** Revogação imediata; modelo simples.
- **Contras:** Introduce estado no servidor; atrito com *statelessness*.
- **Por que rejeitada:** Contraria a API *stateless* replicável ([ADR-0001](#)).

Alternativa 2: OAuth2/OIDC com IdP federado (ex.: Keycloak)

- **Descrição:** Delegar autenticação a um *Identity Provider*.
- **Prós:** SSO e gestão robusta de identidade.
- **Contras:** Infraestrutura adicional fora do escopo do MVP.
- **Por que rejeitada:** *Overkill* para o MVP; fica como evolução.

Consequências

Positivas

- Autenticação *stateless* → escala horizontal sem afinidade de sessão.
- Padrão amplamente suportado; RBAC diretamente nos *claims*.

Negativas

- Revogação antes da expiração exige *blocklist*.
- O cliente precisa proteger os tokens.

Riscos

- Vazamento de token → mitigado por **HTTPS**, **curta expiração**, **rotação** do refresh e **segredo forte** de assinatura.

Notas de Implementação

- `OAuth2PasswordBearer` do FastAPI; assinatura HS256 (ou RS256).
- Access token ~15 min + refresh token de vida mais longa.
- Senhas com *hash* forte (bcrypt/argon2), nunca em texto claro.

Referências

- [arquitetura-c4.md](#) — Seção 6
- [ADR-0003](#) — REST/JSON com OpenAPI
- [ADR-0012](#) — Explicabilidade e auditoria

ADR-0005: PostgreSQL + SQLAlchemy para persistência

Status

Aceito

Data

2026-07-14

Contexto

O sistema precisa persistir **usuários, planos, operadoras, contratos, solicitações, análises e uma trilha de auditoria**. Os dados são majoritariamente **relacionais e transacionais** (entidades com relacionamentos, estados e integridade referencial), com forte requisito de **consistência e rastreabilidade**. Também há conteúdo semiestruturado (respostas da IA, metadados) e necessidade futura de **busca vetorial** (ver [ADR-0007](#)).

Decisão

Usar **PostgreSQL** como banco relacional principal, com transações ACID, acessado via **SQLAlchemy** (ORM/repositórios). Campos semiestruturados usam colunas **JSONB**. O mesmo PostgreSQL hospeda os embeddings vetoriais (pgvector), evitando um segundo banco.

Alternativas Consideradas

Alternativa 1: Banco NoSQL de documentos (ex.: MongoDB)

- **Descrição:** Persistir tudo como documentos.
- **Prós:** Flexível para dados semiestruturados.
- **Contras:** Transações/relacionamentos e integridade referencial mais fracos.
- **Por que rejeitada:** O modelo é relacional e exige consistência forte para auditoria.

Alternativa 2: SQLite

- **Descrição:** Banco embarcado em arquivo.
- **Prós:** Zero operação; ótimo para desenvolvimento.
- **Contras:** Concorrência e escala limitadas.
- **Por que rejeitada:** Inadequado para produção concorrente (útil só em testes).

Consequências

Positivas

- ACID garante consistência das decisões e da auditoria.
- `JSONB` cobre o semiestruturado; **pgvector** cobre a busca semântica — tudo em um só banco.
- Ferramental maduro (migrações, *backup*, réplicas).

Negativas

- Requer operação de banco (serviço gerenciado mitiga).
- Esquema relacional exige migrações disciplinadas.

Riscos

- Crescimento da trilha de auditoria → mitigado por índices, particionamento e política de retenção.

Notas de Implementação

- ORM/Repositórios com SQLAlchemy; migrações com **Alembic** ([ADR-0006](#)).
- Tabela de auditoria **append-only** (sem update/delete).
- Extensão `vector` habilitada no banco ([ADR-0007](#)).

Referências

- [arquitetura-c4.md](#) — Seção 5
- [ADR-0006](#) — Alembic para migrações
- [ADR-0007](#) — pgvector para RAG

ADR-0006: Alembic para versionamento e migração de schema

Status

Aceito

Data

2026-07-14

Contexto

O banco **PostgreSQL** ([ADR-0005](#)) evolui ao longo das *sprints* (cadastros, solicitações, análises, auditoria e embeddings do pgvector). É preciso **versionar** as mudanças de schema de forma **controlada, reproduzível e reversível**, aplicadas de maneira idêntica em vários ambientes (dev/prod), evitando alterações manuais e *drift*. O sistema já usa SQLAlchemy.

Decisão

Usar **Alembic** (companheiro do SQLAlchemy) para **migrações versionadas** de schema. Cada mudança é uma **revisão** no repositório (com `upgrade` e `downgrade`), aplicada de forma **determinística**. As migrações fazem parte do **processo de deploy**.

Alternativas Consideradas

Alternativa 1: SQL manual / scripts ad-hoc

- **Descrição:** Escrever e aplicar SQL de alteração à mão.
- **Prós:** Controle total sobre o SQL.
- **Contras:** Sem histórico versionado; propenso a *drift* e erro humano.
- **Por que rejeitada:** Não rastreável nem reproduzível entre ambientes.

Alternativa 2: Auto-create do ORM (`create_all`)

- **Descrição:** Deixar o SQLAlchemy criar as tabelas a partir dos modelos.
- **Prós:** Rápido no início do desenvolvimento.
- **Contras:** Não gerencia alterações incrementais nem produção.
- **Por que rejeitada:** Inadequado para produção e para a auditabilidade exigida.

Consequências

Positivas

- Histórico de schema **versionado no Git**; revisável em *pull request*.
- Reprodutível entre ambientes; integra-se ao SQLAlchemy.

Negativas

- Exige disciplina para escrever e revisar migrações.
- Migrações de **dados** exigem cuidado extra.

Riscos

- Migração destrutiva em produção → **revisão obrigatória**, **backup** antes e migrações **reversíveis** quando possível.

Notas de Implementação

- `alembic revision --autogenerate`, sempre **revisado manualmente**.
- A migração inicial cria a **extensão** `vector` de forma idempotente.
- Migrações executadas no *pipeline* **antes** de subir a aplicação ([ADR-0011](#)).

Referências

- [arquitetura-c4.md](#) — Seção 5
- [ADR-0005](#) — PostgreSQL + SQLAlchemy

ADR-0007: pgvector para busca semântica das normas da ANS (RAG)

Status

Aceito

Data

2026-07-14

Contexto

Uma capacidade central do MVP é a **consulta das normas da ANS** e o uso da IA para **fundamentar as sugestões** nas normas e nos contratos das operadoras. Para fornecer contexto relevante ao LLM (padrão **RAG — Retrieval-Augmented Generation**) e **reduzir alucinação**, é preciso recuperar trechos de normas e cláusulas **semanticamente** relacionados a cada pedido — não apenas por correspondência de palavras. Isso exige **busca por similaridade** sobre **embeddings vetoriais**. O sistema já usa PostgreSQL ([ADR-0005](#)).

Decisão

Usar a extensão **pgvector** no PostgreSQL para armazenar **embeddings** de normas da ANS e cláusulas contratuais, e realizar **busca por similaridade** (distância de cosseno / produto interno) para montar o contexto do RAG. Tudo permanece no **mesmo banco relacional**, sem introduzir um *vector database* dedicado nesta fase.

Alternativas Consideradas

Alternativa 1: Vector database dedicado (Pinecone, Weaviate, Qdrant)

- **Descrição:** Serviço especializado em busca de vetores.
- **Prós:** Otimizado para escala massiva de vetores.
- **Contras:** Mais um sistema para operar e custear; sincronização com o relacional.
- **Por que rejeitada:** O volume do MVP não justifica; pgvector reaproveita o PostgreSQL.

Alternativa 2: Busca textual (full-text / BM25)

- **Descrição:** Recuperação por índice textual e ranqueamento léxico.
- **Prós:** Simples; não depende de embeddings.
- **Contras:** Não captura similaridade **semântica**; recall pior para linguagem jurídica.
- **Por que rejeitada:** Qualidade do RAG inferior para o domínio.

Consequências

Positivas

- **Um único banco** — dados relacionais e vetores juntos, com ACID.
- Índices ANN (HNSW / IVFFlat) para busca eficiente.
- Base para **explicabilidade**: a sugestão cita as normas recuperadas.

Negativas

- Escala de vetores menor que soluções dedicadas.
- Depende de um modelo/serviço externo para gerar embeddings.

Riscos

- Custo/latência da geração de embeddings → *cache* de embeddings.
- Troca do modelo de embedding altera o espaço vetorial → exige **re-indexação** e **versionamento**.

Notas de Implementação

- Coluna do tipo `vector` ; índice **HNSW**.
- *Pipeline* de ingestão das normas da ANS (extração → *chunking* → embedding → indexação).
- Embeddings gerados via **gateway multiprovedor** ([ADR-0008](#)).

Referências

- [arquitetura-c4.md](#) — Seções 6 e 7
- [ADR-0005](#) — PostgreSQL + SQLAlchemy
- [ADR-0008](#) — LLM multiprovedor

ADR-0008: Integração multiprovedor de LLM (OpenAI/Claude)

Status

Aceito

Data

2026-07-14

Contexto

A análise inteligente do pedido e a geração da sugestão dependem de um **LLM**. O PDF cita **OpenAI/Claude** como opções. LLMs são serviços **externos**, com **latência, custo e disponibilidade variáveis**, e evoluem rápido (novos modelos, mudança de preços). Dependendo de um único provedor cria **risco de lock-in** e de indisponibilidade.

Decisão

Isolar o acesso ao LLM atrás de um **Gateway de LLM** (camada de abstração), com uma interface única e **adaptadores por provedor** (OpenAI e Anthropic Claude). A seleção do provedor/modelo é **configurável**, permitindo *fallback* entre provedores.

Alternativas Consideradas

Alternativa 1: Integração direta com um único provedor

- **Descrição:** Chamar o SDK de um provedor diretamente na lógica de negócio.
- **Prós:** Mais simples e rápido de implementar.
- **Contras:** *Lock-in*; troca de provedor exige mudar código espalhado.
- **Por que rejeitada:** Falta de portabilidade e resiliência.

Alternativa 2: Framework de orquestração pesado

- **Descrição:** Adotar um framework amplo de *orchestration* de LLM.
- **Prós:** Muitos recursos prontos.
- **Contras:** Complexidade e dependência grandes para a necessidade do MVP.
- **Por que rejeitada:** *Overkill*; uma abstração fina é suficiente.

Consequências

Positivas

- Evita *lock-in*; troca de provedor/modelo por configuração.
- **Resiliência**: *fallback* entre provedores em caso de falha.
- Ponto único para *timeouts*, *retry*, *cache* e controle de custo.

Negativas

- Camada extra de abstração para manter.
- Recursos ficam limitados ao **menor denominador comum** entre as APIs.

Riscos

- Diferença de comportamento entre provedores → mitigada por testes e *prompts* parametrizados por adaptador.

Notas de Implementação

- Interface `LLMProvider` com implementações `OpenAIProvider` e `ClaudeProvider`.
- *Timeouts*, *retry* com *backoff* e *fallback* configuráveis.
- Segredos de API protegidos (variáveis de ambiente / cofre de segredos).
- O Gateway também gera **embeddings** para o RAG ([ADR-0007](#)).

Referências

- [arquitetura-c4.md](#) — Seção 6
- [ADR-0009](#) — Validação humana obrigatória

ADR-0009: Validação humana obrigatória (human-in-the-loop)

Status

Aceito

Data

2026-07-14

Contexto

A jornada do usuário prevê: **IA analisa** → **sistema gera sugestão** → **conciliador valida** → **finaliza a conciliação**. A IA é **probabilística** e pode **errar ou alucinar**; a decisão trata de **direitos em saúde** e tem impacto jurídico. Uma das hipóteses do projeto é que **o conciliador confiará nas recomendações da IA** — o que só é aceitável com **controle humano** e uso de **IA responsável**.

Decisão

A sugestão da IA **nunca** é aplicada automaticamente. Toda sugestão nasce no estado **PENDENTE** e só se torna decisão após **validação humana** do conciliador (aprovar, ajustar ou rejeitar). O sistema apoia, mas **não substitui** a decisão humana.

Alternativas Consideradas

Alternativa 1: Decisão automática da IA

- **Descrição:** Aplicar a sugestão sem revisão humana.
- **Prós:** Máxima velocidade e automação.
- **Contras:** Risco jurídico e ético; erros/alucinação sem barreira.
- **Por que rejeitada:** Inaceitável para decisões sobre saúde.

Alternativa 2: Automação parcial por limiar de confiança

- **Descrição:** Auto-aprovar casos simples acima de um *score*.
- **Prós:** Ganho de eficiência em casos triviais.
- **Contras:** Difícil calibrar; risco residual sem histórico validado.
- **Por que rejeitada agora:** Prematuro no MVP; fica como **evolução** com revisão por amostragem.

Consequências

Positivas

- Segurança jurídica e **confiança** do conciliador na ferramenta.
- Gera dados rotulados (sugestão vs. decisão) para avaliar a qualidade da IA.

Negativas

- Mantém uma etapa manual (não é 100% automático).

Riscos

- **Viés de automação** (aceitar sem revisar de fato) → mitigado apresentando a **fundamentação** e exigindo ação explícita ([ADR-0012](#)).

Notas de Implementação

- Estados da sugestão: **PENDENTE** → **APROVADA** / **AJUSTADA** / **REJEITADA** .
- Registrar **sugestão original da IA** e **ação do conciliador** (auditoria).
- Métrica de sucesso: taxa de sugestões aceitas (meta do PDF: > 80%).

Referências

- [arquitetura-c4.md](#) — Seção 7
- [ADR-0008](#) — LLM multiprovedor
- [ADR-0012](#) — Explicabilidade e auditoria

ADR-0010: Armazenamento de documentos e contratos (upload)

Status

Aceito

Data

2026-07-14

Contexto

Funcionalidades do MVP incluem **upload do contrato** e **upload dos documentos** (`POST /upload`). Esses arquivos são **binários** (PDF, imagens), podem ser **grandes**, contêm **dados sensíveis de saúde** e alimentam a análise de IA (texto extraído para o RAG). É preciso decidir **onde** e **como** armazená-los.

Decisão

Armazenar os **arquivos binários em object storage** compatível com **S3** (ex.: MinIO em desenvolvimento/on-prem, bucket gerenciado em produção), guardando no PostgreSQL apenas os **metadados e referências** (chave do objeto, tipo, *hash*, tamanho, vínculo com a solicitação). O texto extraído para o RAG é processado e indexado via pgvector ([ADR-0007](#)).

Alternativas Consideradas

Alternativa 1: Guardar binários no PostgreSQL (BYTEA / Large Objects)

- **Descrição:** Persistir os arquivos dentro do banco.
- **Prós:** Um só sistema; transacional.
- **Contras:** Incha o banco; prejudica *backup*, replicação e desempenho.
- **Por que rejeitada:** Não escala para binários grandes.

Alternativa 2: Sistema de arquivos local do container

- **Descrição:** Salvar no disco do container da API.
- **Prós:** Simples.
- **Contras:** Efêmero, não compartilhado entre réplicas, não durável.
- **Por que rejeitada:** Contraria a API *stateless*/replicável ([ADR-0001](#)).

Consequências

Positivas

- Banco enxuto e rápido; *storage* escalável e durável.
- **URLs pré-assinadas** para acesso controlado; separação metadados × binário.

Negativas

- Mais um serviço (object storage) para operar.
- Consistência eventual entre metadado e objeto.

Riscos

- Acesso indevido a dados sensíveis → **criptografia em repouso, URLs de curta duração** e controle de acesso.
- Órfãos (objeto sem metadado ou vice-versa) → rotina de **reconciliação**.

Notas de Implementação

- Cliente S3 (boto3/aioboto3); validação de **tipo e tamanho** no upload.
- Chave por solicitação; criptografia SSE; retenção conforme sensibilidade do dado.
- *Scan* antivírus opcional antes de processar.

Referências

- [arquitetura-c4.md](#) — Seções 5 e 7
- [ADR-0007](#) — pgvector para RAG
- [ADR-0012](#) — Explicabilidade e auditoria

ADR-0011: Docker para empacotamento e ambiente reprodutível

Status

Aceito

Data

2026-07-14

Contexto

A solução é composta por múltiplos componentes que precisam de ambientes **consistentes** entre desenvolvimento e produção: **API REST**, **PostgreSQL** (com pgvector) e **object storage**. Python exige controle **reprodutível** de dependências e versões. É preciso facilitar o *onboarding* do time e padronizar o *deploy*.

Decisão

Containerizar os componentes com **Docker**. A aplicação tem uma **imagem** própria, com *build* reprodutível a partir do *lockfile*. Usar **Docker Compose** para orquestrar o ambiente local (API, PostgreSQL, object storage).

Alternativas Consideradas

Alternativa 1: Deploy direto no host (virtualenv + systemd)

- **Descrição:** Rodar a aplicação diretamente no SO do servidor.
- **Prós:** Simples; sem camada de container.
- **Contras:** "*funciona na minha máquina*"; *drift* de ambiente.
- **Por que rejeitada:** Reprodutibilidade insuficiente.

Alternativa 2: Buildpacks / PaaS sem Docker explícito

- **Descrição:** Deixar a plataforma empacotar a aplicação.
- **Prós:** Menos configuração inicial.
- **Contras:** Menor controle e transparência; *lock-in* de plataforma.
- **Por que rejeitada:** Menos portátil; preferimos um artefato de container padrão.

Consequências

Positivas

- **Paridade dev/prod**; *onboarding* rápido (`docker compose up`).
- Portável para qualquer orquestrador (ex.: Kubernetes) em produção.
- Base para escalar réplicas da API de forma horizontal.

Negativas

- Curva de aprendizado de Docker/Compose.
- Imagens precisam ser mantidas e atualizadas (segurança).

Riscos

- Imagens com vulnerabilidades → *scan* de imagem, base **slim** e atualização periódica.

Notas de Implementação

- `Dockerfile` **multi-stage**; base *slim*; usuário **não-root**; `HEALTHCHECK` .
- **Docker Compose** para desenvolvimento.
- As migrações ([ADR-0006](#)) rodam como passo do *deploy* antes de subir a aplicação.

Referências

- [arquitetura-c4.md](#) — Seção 5
- [ADR-0002](#) — Python + FastAPI
- [ADR-0006](#) — Alembic para migrações

ADR-0012: Explicabilidade e auditoria das decisões de IA

Status

Aceito

Data

2026-07-14

Contexto

Entre os **diferenciais** do produto estão a **explicabilidade da decisão** e a **padronização**, e a conclusão do projeto destaca **IA responsável**. Além disso, o MVP entrega um **relatório da decisão**. Para que o conciliador **confie** na sugestão e para atender a exigências de **transparência e rastreabilidade** (dados sensíveis de saúde), cada sugestão precisa ser **fundamentada** e cada decisão precisa ser **auditável**.

Decisão

Toda sugestão da IA deve vir acompanhada da sua **fundamentação** — as **normas da ANS** e **cláusulas contratuais** recuperadas (RAG) que a embasam — e ser registrada numa **trilha de auditoria append-only**. A auditoria guarda: entrada, contexto recuperado, **sugestão original da IA**, **provedor/modelo** usado e a **ação do conciliador** (aprovar/ajustar/rejeitar), com autor e carimbo de tempo.

Alternativas Consideradas

Alternativa 1: Registrar apenas a decisão final

- **Descrição:** Persistir só o resultado, sem a sugestão original nem o contexto.
- **Prós:** Menos dados armazenados.
- **Contras:** Impossível auditar divergências ou avaliar a qualidade da IA.
- **Por que rejeitada:** Perde rastreabilidade e explicabilidade.

Alternativa 2: Log de aplicação genérico

- **Descrição:** Depender apenas de logs técnicos.
- **Prós:** Já existe para depuração.
- **Contras:** Não estruturado, volátil, sem garantias de integridade.
- **Por que rejeitada:** Insuficiente como trilha de auditoria de decisões.

Consequências

Positivas

- **Confiança** e transparência: a sugestão mostra em que se baseia.
- Base para **métricas de qualidade** (sugestão vs. decisão humana).
- Rastreabilidade completa de quem decidiu o quê e com base em quê.

Negativas

- Volume de dados de auditoria cresce com o uso.
- Exige cuidado com **dados sensíveis** nos registros.

Riscos

- Exposição de dados sensíveis na trilha → **minimização**, controle de acesso e criptografia; retenção conforme política.

Notas de Implementação

- Tabela de auditoria **append-only** (sem update/delete); **JSONB** para o contexto.
- Guardar **referências** às normas/cláusulas (IDs), não cópias desnecessárias.
- O relatório da decisão é gerado a partir da auditoria.

Referências

- [arquitetura-c4.md](#) — Seções 3 e 7
- [ADR-0007](#) — pgvector para RAG
- [ADR-0009](#) — Validação humana obrigatória